

PRACTICAL WEEK-2

NAME:VARSHITH COMPLE

TASK 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    char *args[20];
    char *token;
    int i = 0;
    pid_t pid;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strlen(command)] = '\0';

    token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command: %s\n", args[0]);

        execvp(args[0], args);

        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```
varshithcomple@VARSHITHs-MacBook-Air ~ % nano t1.c
varshithcomple@VARSHITHs-MacBook-Air ~ % gcc t1.c -o t1
clang: warning: overriding deployment version from '16.0' to '26.0' [-Woverriding-deployment-version]
varshithcomple@VARSHITHs-MacBook-Air ~ % ./t1
Enter a Linux command: ls
Parent Process
Parent PID : 69913
Child PID : 69914
Child Process
Child PID : 69914
Parent PID : 69913
Executing command: ls
Applications  Downloads  first      fork.c     Library    ms.c       notes.c    op.c       Public    task1      tree
Desktop      exec       first.c   is         Movies     Music      notes.txt  OSSP       ti        task1.c   tree.c
Documents    exec.c    fork      is.c       ms         notes      op         Pictures  t1.c     task1.c.save
Child process completed.
varshithcomple@VARSHITHs-MacBook-Air ~ %
```

TASK 2:

```
Processes: 685 total, 2 running, 1 stuck, 682 sleeping, 2992 threads
Load Avg: 2.84, 1.88, 1.77 CPU usage: 2.4% user, 3.53% sys, 94.41% idle SharedLibs: 1836M resident, 199M data, 147M linkedit. MemRegions: 0 total, 0B resident, 0B private, 2578M shared.
```

[illegible]

The CPU is the main processing resource of a computer. The operating system manages access to the CPU using process scheduling. The OS creates and manages processes and decides which process gets CPU time. Commands such as `ps` and `top` allow the user to observe processes and their CPU usage. `lscpu` provides information about the underlying CPU hardware, while the

operating system provides processes with an abstract view of CPU execution rather than requiring them to directly control the processor.

3. Memory Abstraction

Memory is used to store programs and data while they are executing. The operating system manages memory and provides each process with its own address space. This prevents processes from directly interfering with each other's memory. The `top` command can be used to observe memory usage by running processes. Thus, the OS hides the complexity of physical memory management and provides programs with a simpler virtual memory environment.

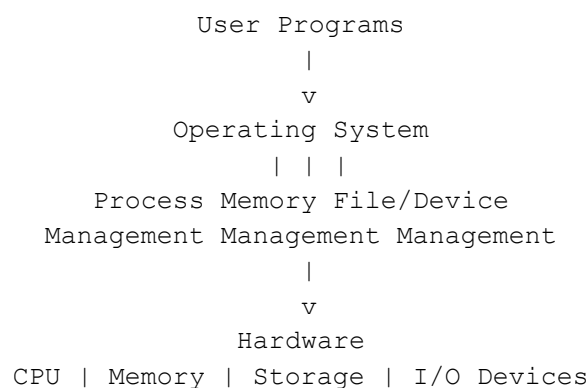
4. Storage Abstraction

Storage devices such as hard disks and SSDs contain permanent data. The operating system abstracts physical storage through files and directories. Instead of applications directly controlling disk sectors, they use operations such as opening, reading, writing, and closing files. The OS manages the actual storage device and performs the required operations on behalf of applications.

5. I/O Device Abstraction

Input/output devices include keyboards, displays, disks, network devices, and other peripherals. The operating system provides interfaces and device drivers that allow applications to communicate with these devices without needing to know their hardware-specific implementation. Therefore, applications can use standard OS services while the operating system handles communication with the actual hardware.

6. Relationship Between Hardware and OS



The operating system acts as an interface between applications and hardware. It manages hardware resources and provides services that applications can use without directly controlling the hardware.

7. Conclusion

The Linux commands `uname`, `lscpu`, `ps`, and `top` provide information about the operating system, CPU, and running processes. The operating system abstracts hardware resources by providing simpler interfaces for CPU execution, memory management, file storage, and I/O operations. This allows applications to use computer resources without directly managing the underlying hardware.

